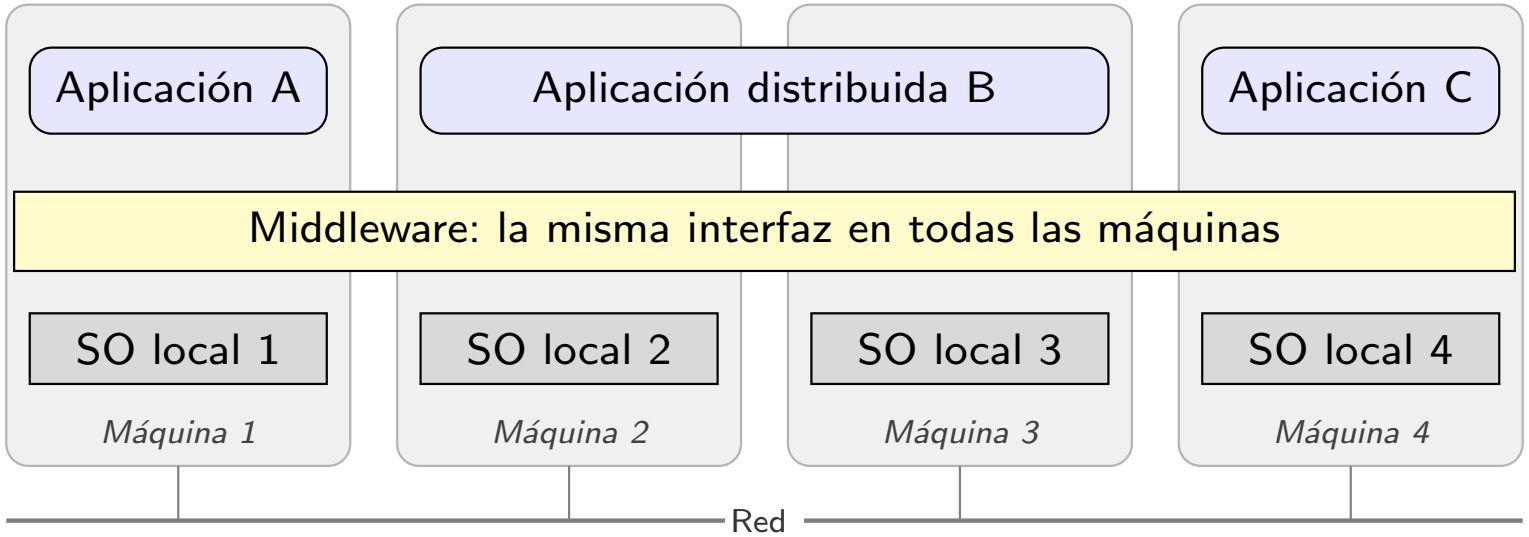
Arquitecturas

Middleware y organización física

Redes de Comunicaciones y Cómputo Distribuido

27 de Octubre de 2026

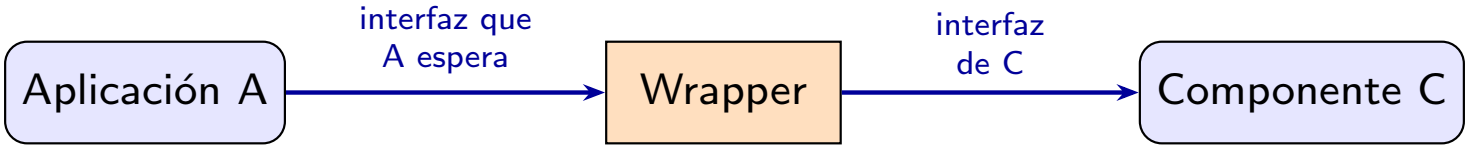
Middleware



- Una capa lógica entre las aplicaciones y los sistemas operativos, que se extiende sobre todas las máquinas.
- Es al sistema distribuido lo que el SO a una computadora: comunicación entre aplicaciones, seguridad y enmascaramiento de fallas.
- Oculta las diferencias de hardware y de sistema operativo.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.13.

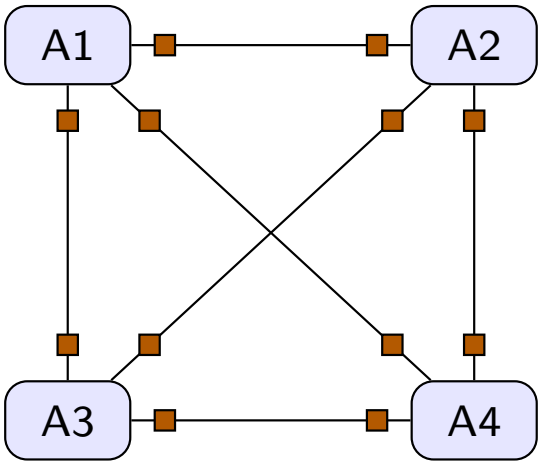
Wrappers



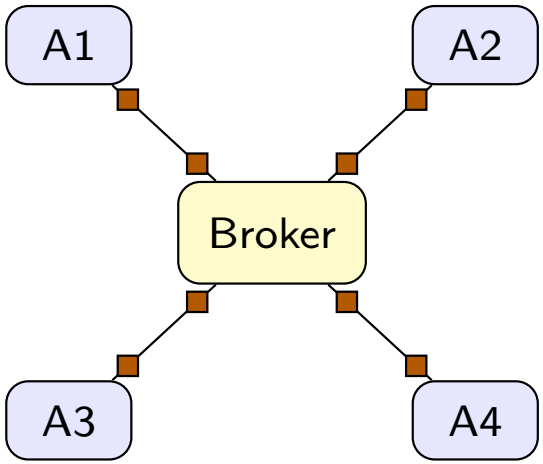
- Un **wrapper** (también llamado *adapter*) le ofrece a una aplicación la interfaz que ella espera, y traduce cada llamada a las funciones que ofrece otro componente.
- Resuelve interfaces incompatibles. Un componente se puede reemplazar sin tocar a quienes lo usan sólo si su interfaz no cambia; si cambia, hace falta un wrapper.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.2.1.

Wrappers vs. broker



(a) Sin broker



(b) Con broker

- ■ = un wrapper.
- **Broker** (también llamado *mediator*): componente lógicamente centralizado por el que pasan todos los accesos. Conoce a todas las aplicaciones, contacta a las que corresponda y combina y transforma las respuestas.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.14.

Ejercicio 5.b y 5.c como ejemplo

Ej 5

b. Si se necesitaran conectar N aplicaciones con diferentes interfaces que se comunican todas contra todas, ¿cuántos wrappers habría que implementar?

c. ¿Qué beneficios trae implementar la solución del inciso anterior usando un broker? Mostrar un ejemplo con un diagrama de conectividad.

Ejercicio 5.b y 5.c como ejemplo

Ej 5

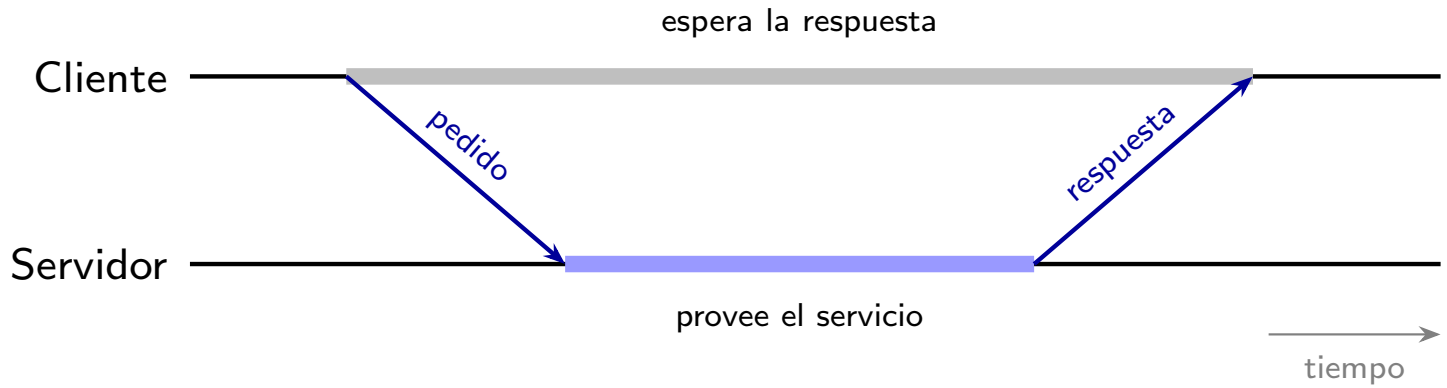
b. Si se necesitaran conectar N aplicaciones con diferentes interfaces que se comunican todas contra todas, ¿cuántos wrappers habría que implementar?

c. ¿Qué beneficios trae implementar la solución del inciso anterior usando un broker? Mostrar un ejemplo con un diagrama de conectividad.

N	2	3	5	20	100
Sin broker: $N(N-1)$	2	6	20	380	9.900
Con broker: $2N$	4	6	10	40	200

- El broker conviene desde $N = 4$: $N(N-1) > 2N \iff N > 3$.
- A cambio, es un punto central: cuello de botella y punto único de falla, y suma un salto en cada llamada.

Arquitectura cliente-servidor



- **Servidor**: proceso que implementa un servicio. **Cliente**: lo pide y espera la respuesta (*request-reply*).
- **Sin conexión** (UDP): eficiente, si la red es confiable, como en una LAN.
- **Con conexión** (TCP): lo habitual en Internet. Armar y cerrar la conexión cuesta, sobre todo con mensajes chicos.
- Un servidor también puede ser cliente de otro servidor.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.16.

¿Y si se pierde un mensaje?

- Sin conexión, lo único que puede hacer el cliente es **reenviar** el pedido si no le llega la respuesta.
- Pero no sabe si se perdió el **pedido** o la **respuesta**. Si se perdió la respuesta, reenviar puede ejecutar la operación **dos veces**.

Operación idempotente

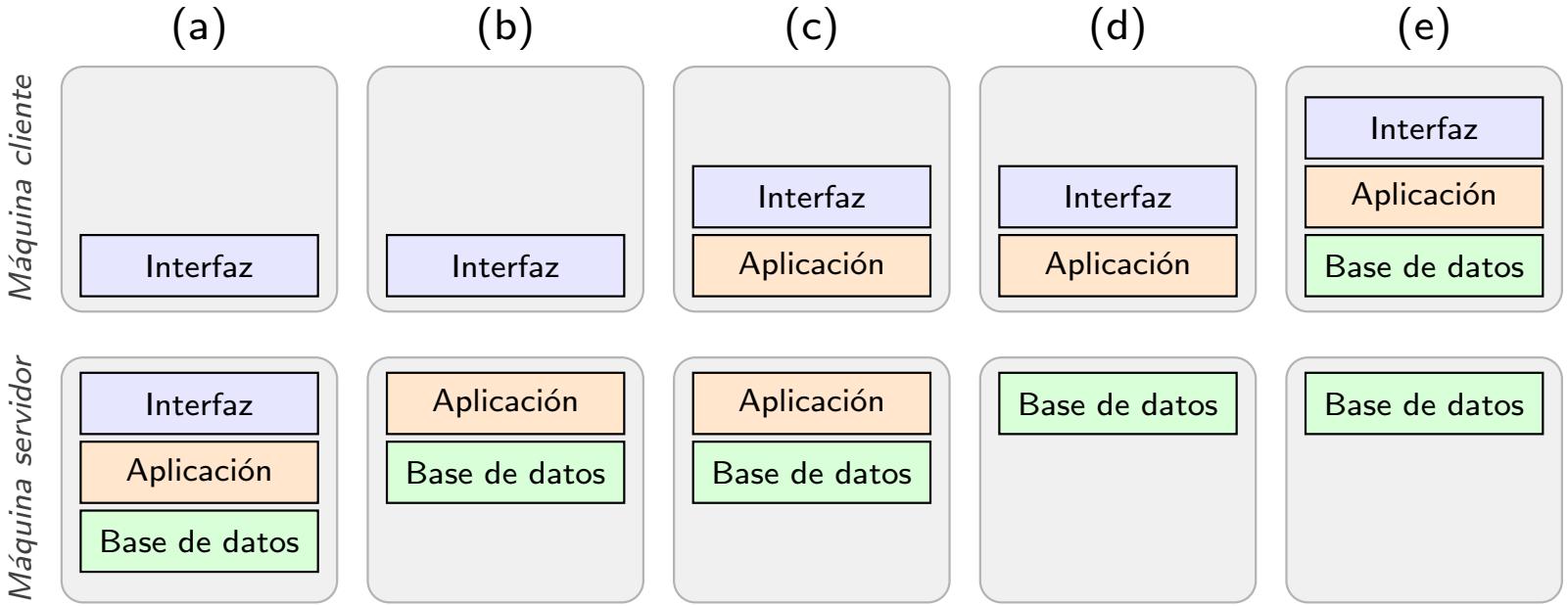
Se puede repetir muchas veces sin hacer daño.

- "¿Cuánto saldo tengo?": idempotente, se puede reenviar.
- "Transferir $10.000": no idempotente, mejor reportar el error.

No hay una solución única: depende de la operación.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.3.1, p. 80.

Dos tiers físicos: ¿dónde se corta?



- Si una capa aparece en las dos máquinas, está partida entre ellas.
- (c): el cliente valida un formulario antes de enviarlo, o un procesador de textos edita en el cliente y corrige la ortografía en el servidor.
- (d): una aplicación bancaria prepara las transacciones en el cliente y las sube a la base del banco.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.17.

Clientes livianos y clientes pesados

- **Livianos** (*thin*), organizaciones (a) a (c): más fáciles de administrar, porque hay poco software en cada usuario y depende poco de su plataforma. A cambio, la interfaz es menos sofisticada.
- **Pesados** (*fat*), organizaciones (d) y (e): siguen teniendo sentido en suites de oficina, en multimedia o para trabajar sin conexión.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Nota 2.5.

Ejercicio 7.1 como ejemplo

Ej 7

Suponiendo una aplicación web two-tiered (cliente-servidor) en la que el cliente envía un formulario con 20 campos obligatorios en los que, para varios de ellos, se deben realizar verificaciones de consistencia (ej.: formato de fechas, fuerza de las passwords, campos sólo numéricos, números de teléfono con área, etc.):

1. Explicar ventajas y desventajas de implementar las verificaciones en el cliente o en el servidor.

Ejercicio 7.1 como ejemplo

Ej 7

Suponiendo una aplicación web two-tiered (cliente-servidor) en la que el cliente envía un formulario con 20 campos obligatorios en los que, para varios de ellos, se deben realizar verificaciones de consistencia (ej.: formato de fechas, fuerza de las passwords, campos sólo numéricos, números de teléfono con área, etc.):

1. Explicar ventajas y desventajas de implementar las verificaciones en el cliente o en el servidor.

	En el cliente	**En el servidor**
Ventajas	Respuesta inmediata; menos pedidos y menos carga en el servidor.	No se puede saltear; una sola implementación, que se actualiza en un solo lugar.
Desventajas	Se puede saltear editando el JavaScript; puede haber campos que no se pueden verificar en el cliente.	Cada error cuesta un viaje de ida y vuelta; más carga en el servidor.

Ejercicio 7.1 como ejemplo

Ej 7

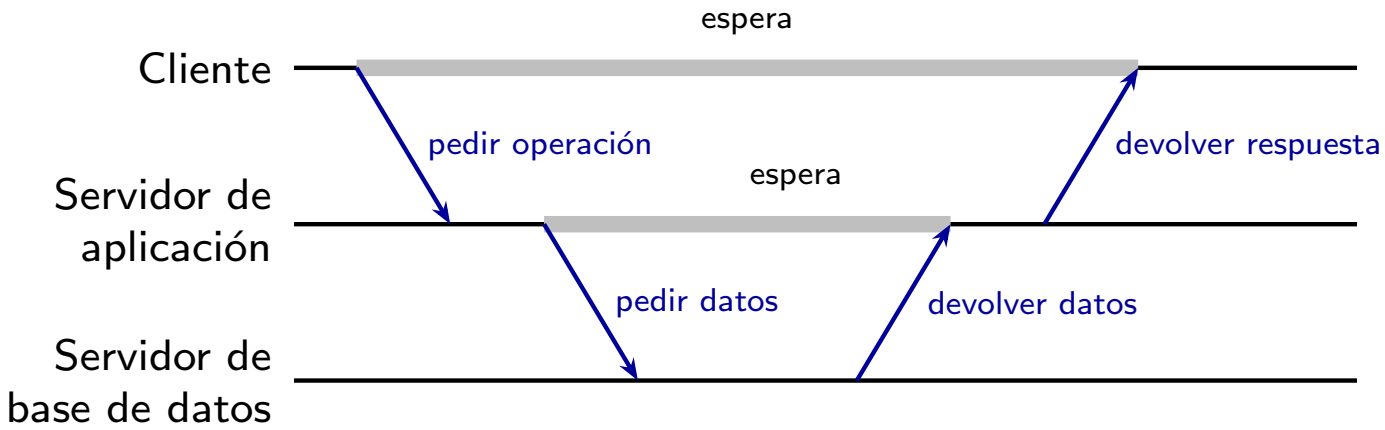
Suponiendo una aplicación web two-tiered (cliente-servidor) en la que el cliente envía un formulario con 20 campos obligatorios en los que, para varios de ellos, se deben realizar verificaciones de consistencia (ej.: formato de fechas, fuerza de las passwords, campos sólo numéricos, números de teléfono con área, etc.):

1. Explicar ventajas y desventajas de implementar las verificaciones en el cliente o en el servidor.

	En el cliente	**En el servidor**
Ventajas	Respuesta inmediata; menos pedidos y menos carga en el servidor.	No se puede saltear; una sola implementación, que se actualiza en un solo lugar.
Desventajas	Se puede saltear editando el JavaScript; puede haber campos que no se pueden verificar en el cliente.	Cada error cuesta un viaje de ida y vuelta; más carga en el servidor.

Conviene validar en los dos lados: en el cliente por usabilidad y en el servidor por seguridad.

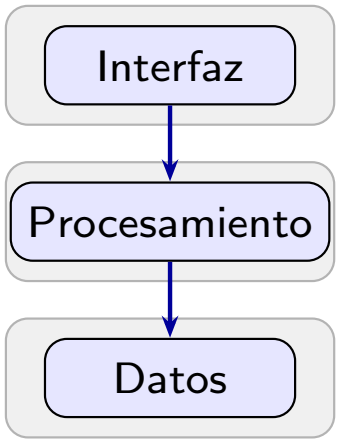
Tres tiers: el servidor también es cliente



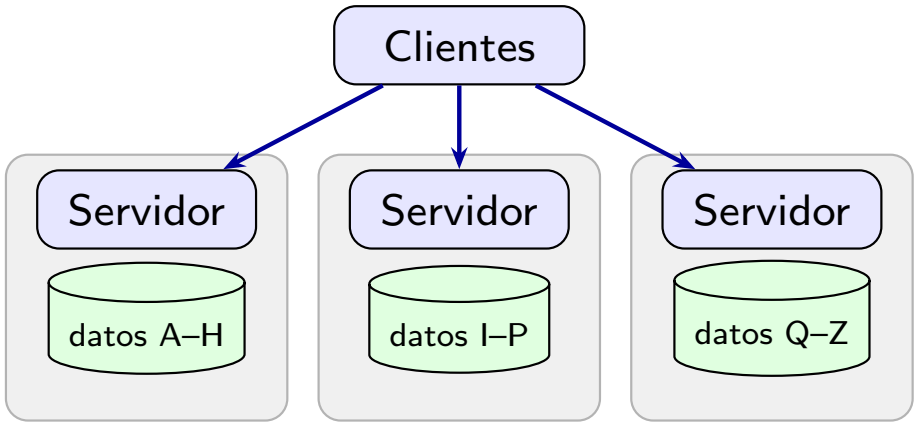
- El procesamiento corre en un servidor aparte, que a su vez es cliente del servidor de datos.
- Ej.: un sitio web. Servidor web $\rightarrow$ servidor de aplicación $\rightarrow$ servidor de base de datos.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.18.

Distribución vertical y horizontal



(a) Vertical: una función distinta en cada máquina

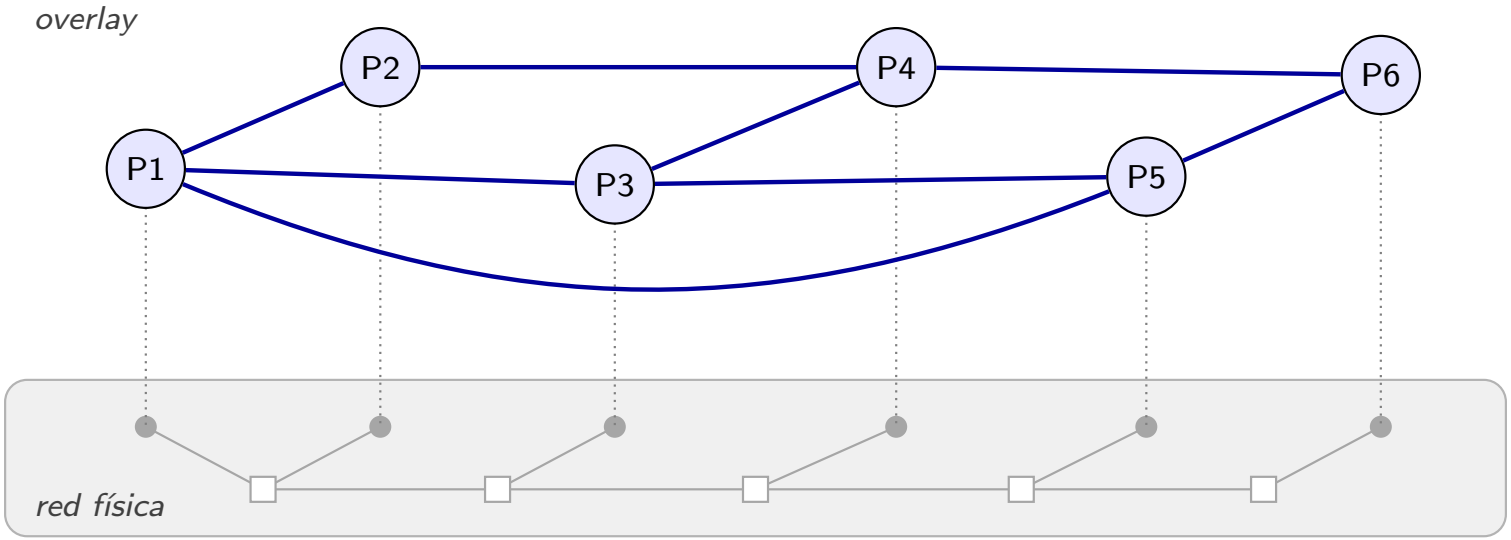


(b) Horizontal: partes equivalentes, cada una con su porción de los datos

- **Vertical**: componentes lógicamente distintos en máquinas distintas. Es lo que hacen las arquitecturas multitier.
- **Horizontal**: un cliente o un servidor partido en partes equivalentes, cada una con su porción de los datos. Reparte la carga.
- Peer-to-peer lleva la distribución horizontal al extremo.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.4, p. 89.

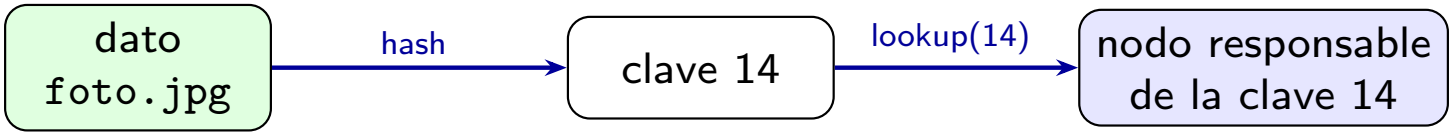
Peer-to-peer: la red overlay



- Todos los procesos son iguales: cada uno es cliente y servidor a la vez.
- **Overlay**: los nodos son procesos y los enlaces, canales posibles (a menudo conexiones TCP). Cada nodo sólo le habla a sus vecinos.
- No es la red física: son procesos y canales, no cables ni radios. Puede ser **estructurada** o **no estructurada**.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.4, p. 89.

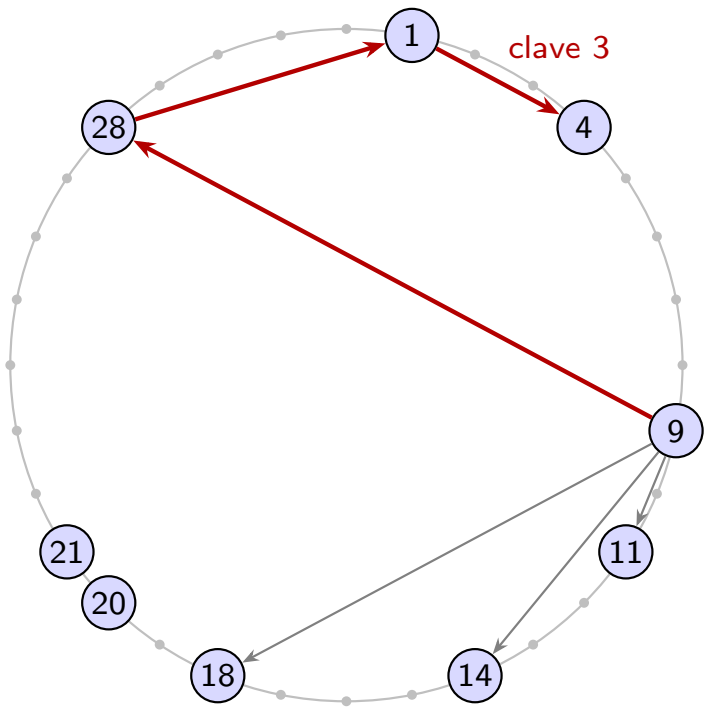
P2P estructurado: tablas de hash distribuidas



- Los nodos forman una topología determinística: un anillo, un árbol, una grilla.
- Cada dato tiene una **clave**: clave = hash(dato).
- Cada nodo tiene un identificador del mismo espacio y guarda los datos de un rango de claves.
- La operación básica es nodo = lookup(clave): el pedido se rutea por la topología hasta el nodo responsable.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.4.1.

Ejemplo: el anillo de Chord



ids de 5 bits: 0 a 31

- La clave k la guarda $\mathrm{succ}(k)$: el primer nodo con id $\geq k$. Ej.: $\mathrm{succ}(7) = \mathrm{succ}(9) = 9$.
- Cada nodo tiene **atajos** a otros nodos. En gris, los del nodo 9.
- Un lookup avanza lo más lejos posible sin pasarse. El 9 busca la clave 3: $9 \to 28 \to 1 \to 4$.
- En general, $O(\log N)$ saltos.
- Al entrar o salir, un nodo sólo intercambia claves con su sucesor.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Nota 2.6.

P2P no estructurado: hay que buscar

- Cada nodo mantiene una lista *ad hoc* de vecinos: la overlay es un grafo aleatorio. Al entrar, pide una lista inicial a un nodo conocido.
- No hay una ruta predeterminada hacia un dato: hay que **buscarlo**.

Flooding

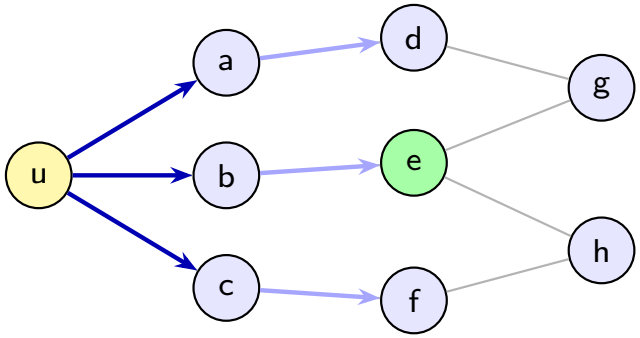
El pedido va a **todos** los vecinos. Quien ya lo vio lo ignora; quien tiene el dato responde y, si no, lo reenvía a sus vecinos. Un **TTL** limita la cantidad de saltos.

Random walk

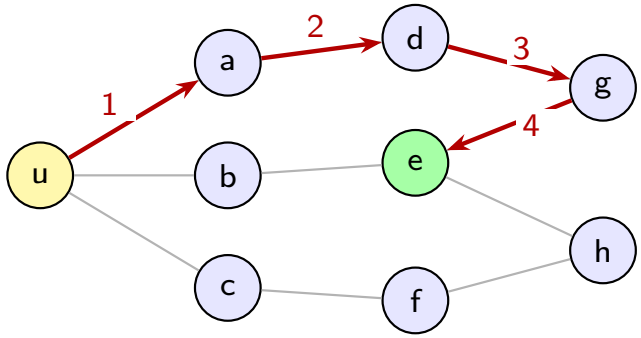
El pedido va a **un** vecino elegido al azar, que hace lo mismo. Mucho menos tráfico, pero tarda más. Con n caminatas en paralelo, el tiempo baja unas n veces.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.4.2.

Flooding vs. random walk



Flooding con TTL 2: 6 mensajes, 2 saltos



Random walk: 4 mensajes, 4 saltos

- u busca el dato que tiene e. En una red grande, el flooding toca muchos más nodos.
- Quien tiene el dato responde con un **aviso** corto (qué tiene, su IP y su puerto): **directo** a quien lo pidió, o **por el camino inverso**, salto a salto.
- Después, quien lo pidió abre una **conexión directa** con ese nodo y baja el dato: los datos no pasan por la overlay.
- En el medio hay políticas: preferir a los vecinos que ya respondieron, o inundar sólo a algunos.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.4.2.

¿Cuántos nodos hay que tocar?

N nodos; cada dato está replicado en r nodos elegidos al azar.

Random walk

Encontrarlo en el intento k:

$$P[k] = \frac{r}{N}\left(1 - \frac{r}{N}\right)^{k-1}$$

S: nodos que hay que consultar, en promedio, hasta encontrarlo:

$$S \approx N/r$$

Flooding a d vecinos

Después de k pasos alcanza a lo sumo

$$R(k) = d\,(d-1)^{k-1}$$

nodos. Lo encuentra cuando

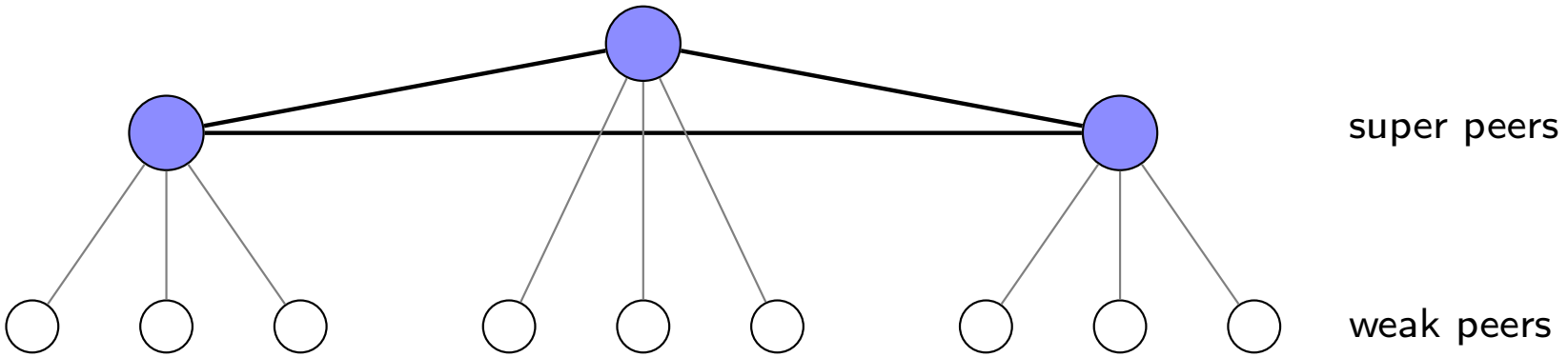
$$\frac{r}{N}\,R(k) \geq 1.$$

Ej.: una red de 100.000 nodos, cada dato en 100 de ellos ($r/N = 0{,}1\,\%$) y $d = 10$ vecinos por nodo.

- Random walk: $S \approx N/r = 1000$ nodos, uno detrás de otro, así que tarda más en responder.
- Flooding: hace falta $R(k) \geq 1000$. Con $R(3) = 810$ no alcanza; con $R(4) = 7290$, sí: 4 pasos y unos 7290 nodos.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Nota 2.7.

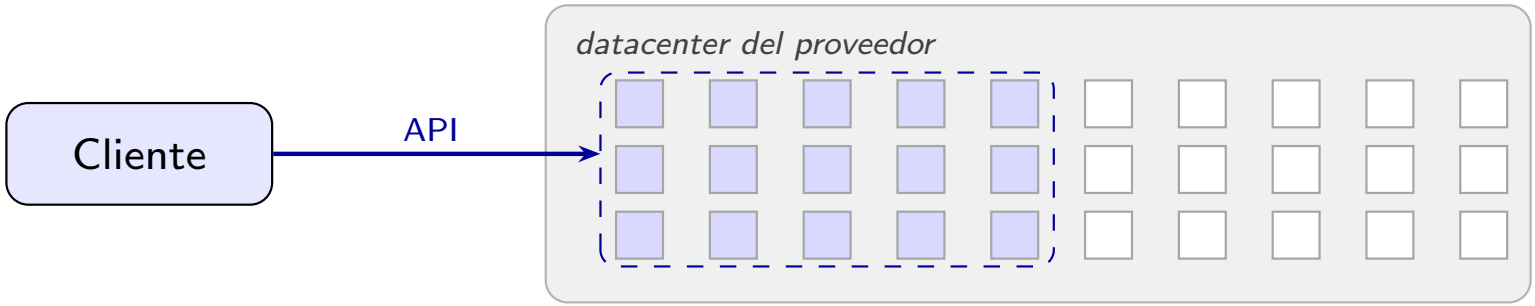
P2P jerárquico: super peers



- Nodos especiales que mantienen un **índice** de los datos o hacen de **broker**.
- Cada weak peer se conecta como cliente a un super peer, y todo su tráfico pasa por él.
- Tienen que ser procesos estables y muy disponibles; se pueden emparejar para tener respaldo.
- ¿Quién es super peer? Es un problema de elección de líder.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.25.

Cloud computing

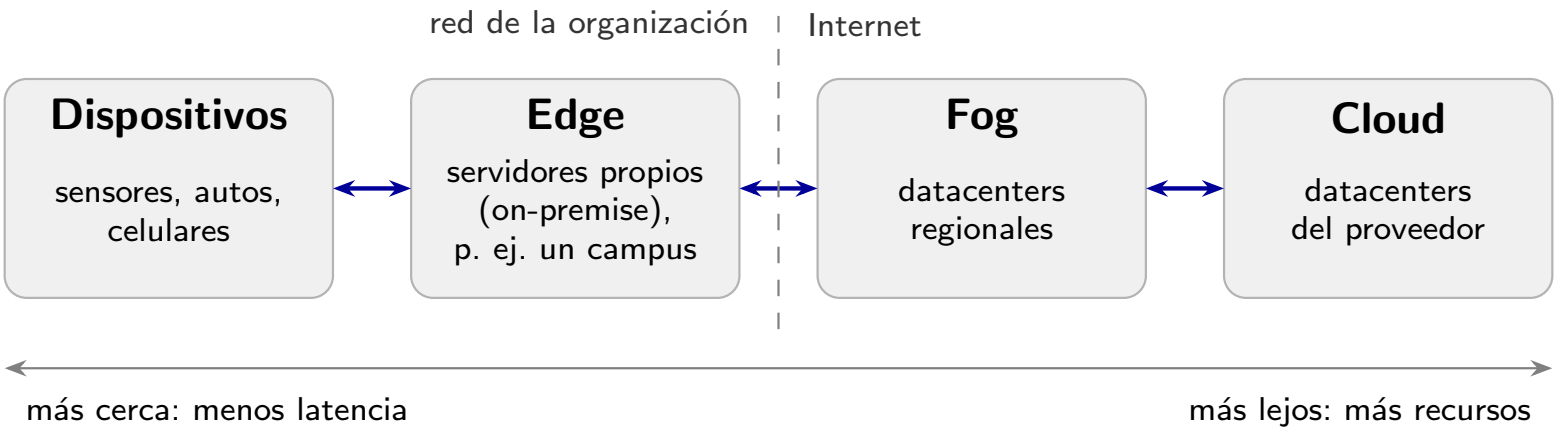


el servidor: oculto y repartido en muchas máquinas

- Un **pool de recursos virtualizados** en datacenters, que se configura dinámicamente: si hay más trabajo, se piden más recursos. Se paga por uso.
- Se ofrece como infraestructura (**IaaS**: máquinas virtuales, almacenamiento), plataforma (**PaaS**: una API para subir y correr programas) o software (**SaaS**: aplicaciones como Gmail o Google Docs).
- Como arquitectura, es un cliente-servidor avanzado: el cliente no sabe dónde está el servidor, que suele estar repartido en todo un datacenter.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.5.1.

Edge, fog y cloud



- **Edge**: servicios en el borde de la red, entre la red de la organización e Internet. Ej.: un campus con servidores propios; gran parte del tráfico de esos servicios nunca sale del campus.
- **Fog**: infraestructura intermedia, más cerca de la nube; p. ej. datacenters regionales que comparten varias organizaciones.
- Los límites entre edge, fog y cloud son difusos.

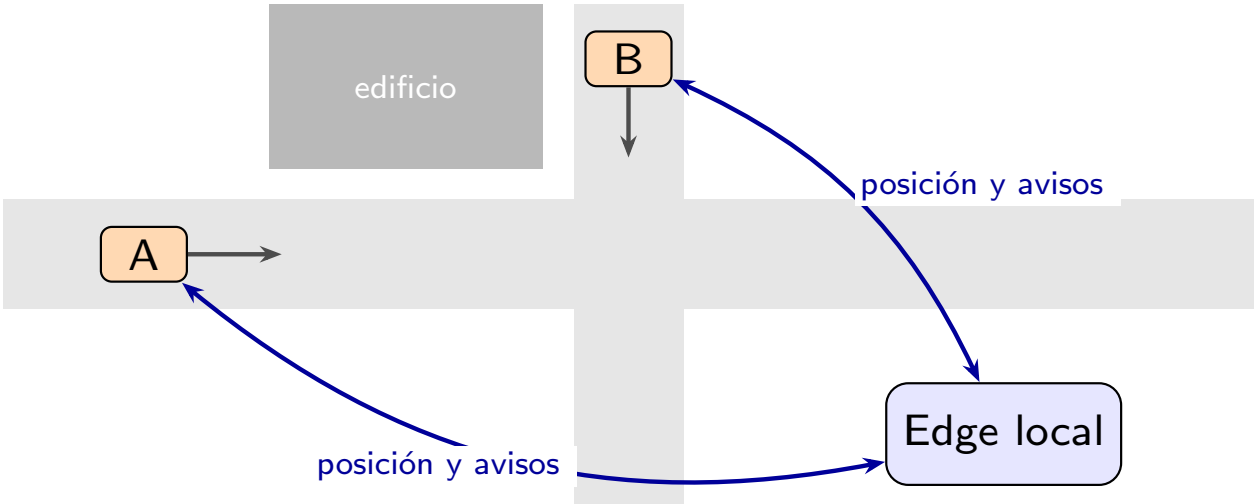
Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.28.

¿Por qué no ir directo a la nube?

- **Latencia**: llegar a la nube puede llevar unos 100 ms, y para coordinar un auto autónomo eso es inaceptable. Es el argumento más fuerte.
- **Ancho de banda**: el video necesita la fuente cerca para dar garantías de calidad (menos jitter).
- **Confiabilidad**: entornos críticos, como hospitales o fábricas, no pueden depender de la conexión con la nube.
- **Regulación**: hay datos que no pueden salir de la organización, como las historias clínicas.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.5.2, pp. 101–103.

Ejemplo: autos que se acercan a un cruce



- Un auto autónomo observa su entorno con sensores y reacciona en tiempo real: coordinar sus movimientos a través de la nube no es aceptable.
- Para detectarse más allá de lo que ven sus sensores, cada auto le pasa su posición a una infraestructura edge local, y así los autos se dan a conocer entre sí al acercarse al cruce.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.5.2, p. 102.

Orquestación: qué va dónde

En la nube sobran recursos; en el edge son escasos y heterogéneos, y la carga cambia mucho. Hay que decidir:

- **Asignación de recursos**: garantizar a cada servicio la CPU, la memoria, el almacenamiento y la red que necesita.
- **Ubicación de servicios**: cuándo y dónde instalar cada uno. Con aplicaciones móviles importa usar el edge más cercano al dispositivo.
- **Selección del edge**: cuál usar. El más cercano no siempre es el mejor; por ejemplo, por su conexión con la nube.

Criterio: cada servicio va en la capa que cumple con la latencia que tolera y que ve la parte del sistema que necesita.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.5.2, pp. 103–104.

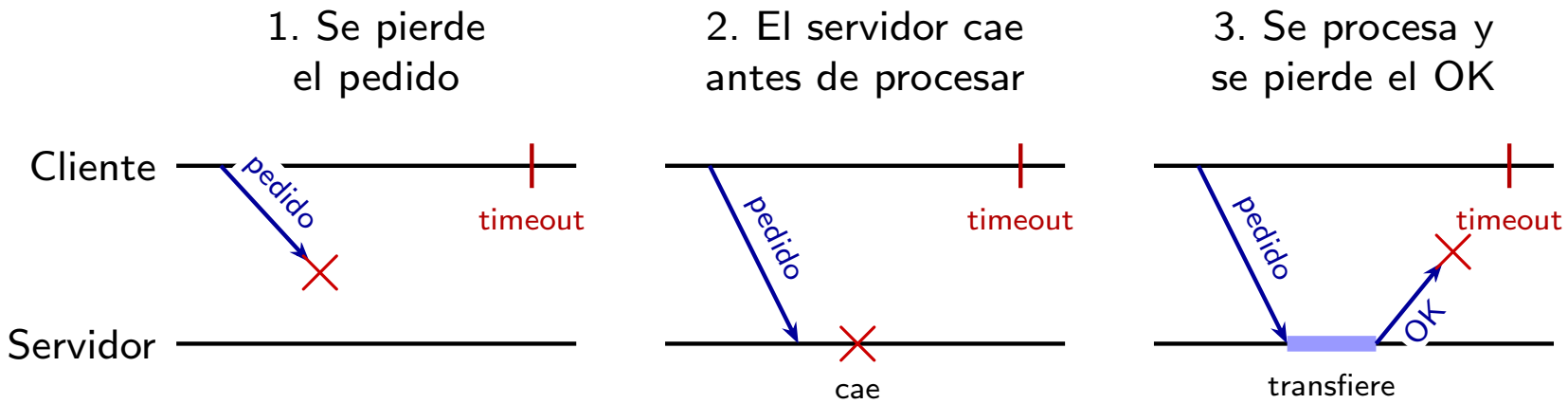
Ejercicio 6:

Supongamos que usamos una aplicación para realizar transferencias bancarias sobre Internet que usa como servicio de transporte UDP. La aplicación cliente envía mensajes con la siguiente estructura: (CuentaOrigen, CuentaDestino, cantidad) y el servidor responde un mensaje 'OK' o 'Failed', indicando si fue posible o no realizar la transferencia.

Explicar qué sucede ante la pérdida de un mensaje del servidor que indica 'OK'.

Supuesto: el cliente espera la respuesta con un *timeout* y puede retransmitir.

Ejercicio 6: el cliente ve siempre lo mismo



En los tres casos el cliente ve lo mismo: un **timeout**. No puede saber qué pasó.

Ejercicio 6: ¿retransmitir?

- **Si retransmite**: en el caso 3 la transferencia se ejecuta **dos veces**.

Ejercicio 6: ¿retransmitir?

- **Si retransmite**: en el caso 3 la transferencia se ejecuta **dos veces**.
- **Si no retransmite**: en los casos 1 y 2 la transferencia **nunca** se ejecuta, y el cliente no lo sabe.

Ejercicio 6: ¿retransmitir?

- **Si retransmite**: en el caso 3 la transferencia se ejecuta **dos veces**.
- **Si no retransmite**: en los casos 1 y 2 la transferencia **nunca** se ejecuta, y el cliente no lo sabe.
- El problema de fondo: transferir **no es idempotente**.

Ejercicio 6: soluciones

Identificador de transacción

El cliente numera cada pedido y el servidor guarda los números que ya atendió. Ante un reintento no repite la transferencia: reenvía la respuesta. Se ejecuta **a lo sumo una vez**.

¿Hasta cuándo guarda el servidor esos números?

Ejercicio 6: soluciones

Identificador de transacción

El cliente numera cada pedido y el servidor guarda los números que ya atendió. Ante un reintento no repite la transferencia: reenvía la respuesta. Se ejecuta **a lo sumo una vez**.

¿Hasta cuándo guarda el servidor esos números?

Hacer idempotente la operación

En lugar de "transferir \$100", "crear la transferencia 7 por \$100". Si el pedido llega dos veces, la segunda no hace nada.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §8.3.2, p. 513.

Ejercicio 6: ¿y con TCP?

- No alcanza: si la conexión se corta después de transferir y antes del OK, vuelve la misma ambigüedad.
- Transporte confiable no es lo mismo que operación confiable.

Ejercicio 9:

Un archivo tar.gz de 1 TB se distribuye dividiéndolo en piezas de 100 MB y se almacena en 100 servidores que almacenan 1000 piezas cada uno. Cada servidor mantiene una lista de 10 vecinos activos con los que intercambia mensajes de pedidos de partes. Suponer que el tiempo de propagación entre los nodos es de 100 ms y que la velocidad de transmisión entre los nodos es de 1 Gbps.

1. ¿Cuánto tiempo, en promedio, tardaría un nodo nuevo que se agrega en la red en obtener una versión entera del archivo tar.gz mediante inundación (*flooding*)?
2. ¿El tiempo bajaría si se usara una caminata al azar (*random walk*)?
3. ¿Y si se usara un broker como super peer?

Ejercicio 9: datos y modelo

- 1 TB / 100 MB = 10.000 piezas.
- 100 servidores $\times$ 1000 piezas = 100.000 piezas guardadas: cada pieza está en **10 servidores**, el 10 % de la red.
- Transmitir una pieza: 100 MB = 800 Mb, que a 1 Gbps tardan **0,8 s**.

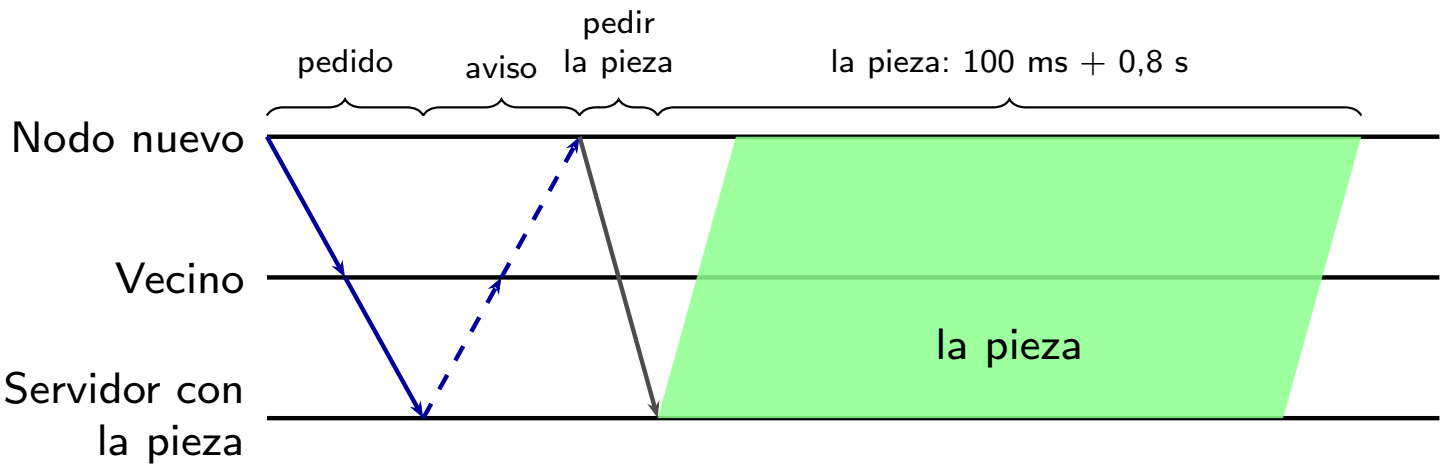
Ejercicio 9: datos y modelo

- 1 TB / 100 MB = 10.000 piezas.
- 100 servidores × 1000 piezas = 100.000 piezas guardadas: cada pieza está en **10 servidores**, el 10 % de la red.
- Transmitir una pieza: 100 MB = 800 Mb, que a 1 Gbps tardan **0,8 s**.

Modelo: listar asunciones antes de hacer las cuentas

- Cada pieza está en 10 servidores elegidos al azar.
- El nodo nuevo busca y baja las piezas de a una.
- Un mensaje corto entre dos servidores tarda 100 ms: sólo cuenta la propagación.
- El pedido recorre h saltos por la overlay, y el aviso de quién tiene la pieza vuelve por el mismo camino.
- Después, el nodo nuevo le pide la pieza directamente a ese servidor, que se la manda a 1 Gbps.

Ejercicio 9: el tiempo de una pieza



Azul: por la overlay (h saltos de ida, h de vuelta; en el dibujo, $h = 2$). Gris: pedido directo de la pieza. En total, $2h + 2$ mensajes, uno detrás de otro.

$$T = \underbrace{2h \cdot 100\ \text{ms}}_{\text{pedido y aviso}} + \underbrace{100\ \text{ms}}_{\text{pedir la pieza}} + \underbrace{100\ \text{ms} + 0{,}8\ \text{s}}_{\text{la pieza}}$$
$$= (2h + 2) \cdot 100\ \text{ms} + 0{,}8\ \text{s}$$

Ejercicio 9: ¿cuántos saltos hacen falta?

Con las fórmulas de la slide 18: $N = 100$ servidores, $r = 10$ copias de cada pieza ($r/N = 0{,}1$) y $d = 10$ vecinos.

- **Flooding**:
  - Paso 1: $R(1) = d = 10$ vecinos. Como $\frac{r}{N} R(1) = 1$, en promedio uno tiene la pieza, pero ninguno la tiene con probabilidad $(1 - \frac{r}{N})^d = 0{,}9^{10} \approx 0{,}35$.
  - Paso 2: $R(2) = d\,(d-1) = 90$ pedidos más. Los vecindarios se superponen, así que llegan a menos de 90 nodos distintos; pero $\frac{r}{N} R(2) = 9$ está muy por encima de 1: casi seguro la encuentra.

$$E[h] = 0{,}65 \cdot 1 + 0{,}35 \cdot 2 = 1{,}35$$

Ejercicio 9: ¿cuántos saltos hacen falta?

Con las fórmulas de la slide 18: $N = 100$ servidores, $r = 10$ copias de cada pieza ($r/N = 0{,}1$) y $d = 10$ vecinos.

- **Flooding**:
  - Paso 1: $R(1) = d = 10$ vecinos. Como $\frac{r}{N} R(1) = 1$, en promedio uno tiene la pieza, pero ninguno la tiene con probabilidad $(1 - \frac{r}{N})^d = 0{,}9^{10} \approx 0{,}35$.
  - Paso 2: $R(2) = d\,(d-1) = 90$ pedidos más. Los vecindarios se superponen, así que llegan a menos de 90 nodos distintos; pero $\frac{r}{N} R(2) = 9$ está muy por encima de 1: casi seguro la encuentra.

$$E[h] = 0{,}65 \cdot 1 + 0{,}35 \cdot 2 = 1{,}35$$

- **Random walk**: $S \approx N/r = 100/10 = 10$ saltos.

Ejercicio 9: ¿cuántos saltos hacen falta?

Con las fórmulas de la slide 18: $N = 100$ servidores, $r = 10$ copias de cada pieza ($r/N = 0{,}1$) y $d = 10$ vecinos.

- **Flooding**:
  - Paso 1: $R(1) = d = 10$ vecinos. Como $\frac{r}{N} R(1) = 1$, en promedio uno tiene la pieza, pero ninguno la tiene con probabilidad $(1 - \frac{r}{N})^d = 0{,}9^{10} \approx 0{,}35$.
  - Paso 2: $R(2) = d\,(d-1) = 90$ pedidos más. Los vecindarios se superponen, así que llegan a menos de 90 nodos distintos; pero $\frac{r}{N} R(2) = 9$ está muy por encima de 1: casi seguro la encuentra.

$$E[h] = 0{,}65 \cdot 1 + 0{,}35 \cdot 2 = 1{,}35$$

- **Random walk**: $S \approx N/r = 100/10 = 10$ saltos.
- **Super peer**: $h = 1$, porque el broker sabe quién tiene cada pieza.

Ejercicio 9: comparación

Estrategia	**Saltos h**	**Mensajes**	**Por pieza**	**Total**
Flooding (TTL 2)				
Random walk				
Super peer (broker)				

- Mensajes por pieza, uno detrás de otro: h pedidos y h avisos por la overlay; después, el pedido directo de la pieza y la pieza. En total, $2h + 2$: con flooding son 4 o 6, según la pieza esté a 1 o 2 saltos, y 4,7 en promedio.
- Cada mensaje tarda 100 ms, y la pieza suma 0,8 s de transmisión a 1 Gbps: $T = (2h + 2) \cdot 100 \text{ ms} + 0{,}8 \text{ s}$.
- El total es por las 10.000 piezas.

Ejercicio 9: comparación

Estrategia	**Saltos h**	**Mensajes**	**Por pieza**	**Total**
Flooding (TTL 2)	1,35 (1 o 2)	4,7	$\approx$ 1,27 s	$\approx$ 3,5 h
Random walk				
Super peer (broker)				

- Mensajes por pieza, uno detrás de otro: h pedidos y h avisos por la overlay; después, el pedido directo de la pieza y la pieza. En total, $2h + 2$: con flooding son 4 o 6, según la pieza esté a 1 o 2 saltos, y 4,7 en promedio.
- Cada mensaje tarda 100 ms, y la pieza suma 0,8 s de transmisión a 1 Gbps: $T = (2h + 2) \cdot 100 \text{ ms} + 0{,}8 \text{ s}$.
- El total es por las 10.000 piezas.

Ejercicio 9: comparación

Estrategia	**Saltos h**	**Mensajes**	**Por pieza**	**Total**
Flooding (TTL 2)	1,35 (1 o 2)	4,7	$\approx$ 1,27 s	$\approx$ 3,5 h
Random walk	$\approx$ 10	$\approx$ 22	$\approx$ 3,0 s	$\approx$ 8,3 h
Super peer (broker)				

- Mensajes por pieza, uno detrás de otro: h pedidos y h avisos por la overlay; después, el pedido directo de la pieza y la pieza. En total, $2h + 2$: con flooding son 4 o 6, según la pieza esté a 1 o 2 saltos, y 4,7 en promedio.
- Cada mensaje tarda 100 ms, y la pieza suma 0,8 s de transmisión a 1 Gbps: $T = (2h + 2) \cdot 100 \text{ ms} + 0{,}8 \text{ s}$.
- El total es por las 10.000 piezas.

Ejercicio 9: comparación

Estrategia	**Saltos h**	**Mensajes**	**Por pieza**	**Total**
Flooding (TTL 2)	1,35 (1 o 2)	4,7	$\approx$ 1,27 s	$\approx$ 3,5 h
Random walk	$\approx$ 10	$\approx$ 22	$\approx$ 3,0 s	$\approx$ 8,3 h
Super peer (broker)	1	4	$\approx$ 1,2 s	$\approx$ 3,3 h

- Mensajes por pieza, uno detrás de otro: h pedidos y h avisos por la overlay; después, el pedido directo de la pieza y la pieza. En total, $2h + 2$: con flooding son 4 o 6, según la pieza esté a 1 o 2 saltos, y 4,7 en promedio.
- Cada mensaje tarda 100 ms, y la pieza suma 0,8 s de transmisión a 1 Gbps: $T = (2h + 2) \cdot 100 \text{ ms} + 0{,}8 \text{ s}$.
- El total es por las 10.000 piezas.

Ejercicio 9: respuestas

1. Con flooding: unos 1,27 s por pieza, **unas 3,5 h** en total.
2. Con random walk el tiempo **no baja**: sube a unas 8,3 h, porque el pedido y el aviso recorren unos 10 saltos cada uno.
3. Con un broker como super peer, unas 3,3 h: un poco menos que con flooding. A cambio, el broker es un punto central.

¿Preguntas?

